

Genetic Algo

```
import random # bring in the "random" toolbox so we can pick random numbers
```

```
# -----
```

```
# STEP 1: Make ONE random chromosome (one possible board)
```

```
# -----
```

```
def generateChrom(size):
```

```
    chrom = []          # start with an empty list -> this will be our chromosome (the board)
```

```
    for i in range(size):    # repeat "size" times (once for each queen/column)
```

```
        r = random.randint(1, size) # pick a random row number between 1 and size for this queen
```

```
        chrom.append(r)      # add that row number to our chromosome list
```

```
    return chrom           # give back the finished chromosome
```

```
# -----
```

```
# STEP 2: Make MANY chromosomes -> this is our "population"
```

```
# -----
```

```
def genPOP(number, size):
```

```
    population = []      # empty list that will hold all the chromosomes
```

```
    for i in range(number):    # repeat "number" times (once for each chromosome we want)
```

```
        pop = generateChrom(size) # create one random chromosome
```

```
        population.append(pop)   # add it to the population list
```

```
    return population        # give back the full population
```

```
# -----
```

```
# STEP 3: Combine two parent chromosomes to make two children
```

```
# -----
```

```
def crossOver(chrom1, chrom2, size):
```

```

k = random.randint(1, size - 2)      # pick a random "cut point" (not at the very start/end)

print("cross over point", k)          # show the cut point for information

child1 = chrom1[:k] + chrom2[k:]      # first part from parent1, second part from parent2
child2 = chrom2[:k] + chrom1[k:]      # first part from parent2, second part from parent1

return child1, child2                # give back both new children


# -----

# STEP 4: Randomly change one gene (queen's row) in a chromosome
# -----

def mutate(chrom, size):

    index = random.randint(0, size - 1) # pick a random position in the chromosome to change
    val = random.randint(1, size)       # pick a new random row value
    chrom[index] = val                   # replace the old value at that position with the new value
    return chrom                        # give back the mutated chromosome


# -----

# STEP 5: Measure how "good" a chromosome is (fewer attacks = better)
# -----

def fitness(chrom, size):

    count_attack = 0                    # start counting attacking pairs of queens at 0
    max_attack = (size * (size - 1)) / 2 # this is the maximum possible number of attacking pairs
    for i in range(size):               # go through every queen...
        for j in range(i + 1, size):    # ...and compare it with every queen after it
            if chrom[i] == chrom[j]:    # if two queens are on the same row, they attack each other
                count_attack += 1        # so we count one attack
    return max_attack - count_attack     # good boards have HIGH fitness (few attacks), so subtract

```

```
# -----
```

```
# STEP 6: Pick the best chromosome out of the whole population
```

```
# -----
```

```
def getBest(population, size):
```

```
    best = population[0]          # assume the first chromosome is the best, for now
    bestFit = fitness(best, size)  # calculate its fitness score
    for chrom in population:      # look through every chromosome in the population
        f = fitness(chrom, size)   # calculate this chromosome's fitness
        if f > bestFit:            # if it's better than our current best...
            best = chrom           # ...remember this chromosome instead
            bestFit = f            # ...and remember its fitness score
    return best, bestFit           # give back the best chromosome and its score
```

```
# -----
```

```
# STEP 7: Pick a parent using fitness — better boards are more
```

```
#     likely to be chosen (this is called "selection")
```

```
# -----
```

```
def selectParent(population, size):
```

```
    tournamentSize = 3            # we'll randomly pick 3 chromosomes and choose the best of them
    candidates = random.sample(population, tournamentSize) # randomly grab 3 different chromosomes from the
    population
    best = candidates[0]           # assume the first one is best, for now
    bestFit = fitness(best, size)   # get its fitness
    for c in candidates:           # check the other candidates
        f = fitness(c, size)        # get this candidate's fitness
        if f > bestFit:             # if it's better...
            best = c                # ...keep this one instead
    bestFit = f
```

```

return best                                # return the winning parent

# -----

# STEP 8: The main Genetic Algorithm loop that solves N-Queens
# -----

def geneticAlgorithm(size, popSize=50, generations=1000, mutationRate=0.2):
    population = genPOP(popSize, size)      # create the first random population
    maxFitness = (size * (size - 1)) / 2    # this is the fitness value of a PERFECT solution (0 attacks)

    for gen in range(generations):          # repeat for a set number of generations
        best, bestFit = getBest(population, size) # find the best chromosome in the current population

        if bestFit == maxFitness:           # if we found a perfect solution (no attacking queens)...
            print(f"Solution found in generation {gen}: {best}") # announce it
            return best                     # ...stop and return it immediately

    newPopulation = []                      # this will hold the next generation
    while len(newPopulation) < popSize:     # keep making children until the new population is full
        parent1 = selectParent(population, size) # pick parent 1 using tournament selection
        parent2 = selectParent(population, size) # pick parent 2 the same way
        child1, child2 = crossOver(parent1, parent2, size) # breed them to get two children

        if random.random() < mutationRate: # with some probability...
            child1 = mutate(child1, size)    # ...mutate the first child
        if random.random() < mutationRate: # with some probability...
            child2 = mutate(child2, size)    # ...mutate the second child

    newPopulation.append(child1)             # add first child to the new population

```

```

        if len(newPopulation) < popSize:      # only add the second child if there's still room
            newPopulation.append(child2)

    population = newPopulation                # replace old population with the new generation

# if we run out of generations without a perfect solution, return the best one found
best, bestFit = getBest(population, size)
print(f"Best solution found (not perfect): {best} with fitness {bestFit} out of {maxFitness}")
return best

# -----
# STEP 9: Run everything (this only runs when the file is executed directly)
# -----
if __name__ == "__main__":
    N = 8                                    # size of the board / number of queens (change this as needed)
    print("Running Genetic Algorithm to solve", N, "-Queens problem...")
    solution = geneticAlgorithm(N)           # run the full genetic algorithm
    print("Final board (row for each column):", solution) # print the final answer

```

Neural NET

```

# 1. IMPORT LIBRARIES

# =====

# Pandas is a tool for manipulating tabular data (rows and columns).
# We use it to load and handle our dataset easily.
import pandas as pd

# This function splits our dataset into training and testing portions.

```

We need it so we can train the model on one set and test it on unseen data.

```
from sklearn.model_selection import train_test_split
```

MinMaxScaler scales numbers to a specific range, usually between 0 and 1.

Neural networks train much better and faster when input values are on a uniform scale.

```
from sklearn.preprocessing import MinMaxScaler
```

LabelEncoder converts text/categorical labels into numeric integers (e.g., "Male"/"Female" to 0/1).

Machine learning models only understand numbers, not plain text.

```
from sklearn.preprocessing import LabelEncoder
```

TensorFlow is Google's core framework for building and training deep learning models.

```
import tensorflow as tf
```

Sequential lets us build a neural network layer-by-layer in a straight sequence.

```
from tensorflow.keras.models import Sequential
```

Dense represents a fully-connected layer where every neuron connects to all neurons in the previous layer.

```
from tensorflow.keras.layers import Dense
```

Adam is an advanced optimization algorithm used to adjust model weights to reduce errors quickly.

```
from tensorflow.keras.optimizers import Adam
```

```
# =====
```

```
# 2. LOAD AND INSPECT DATA
```

```
# =====
```

```
# Load the dataset from a CSV file into a Pandas DataFrame named 'df'.
```

```

# This brings the raw dataset into memory for analysis.

df = pd.read_csv('/kaggle/input/datasets/sharanshetty79/gym-membership-
prediction/gym_membership_prediction_with_professional_status.csv')

# Print the loaded dataset to inspect rows, columns, and data types in the console.

print(df)


# =====

# 3. DATA PREPROCESSING & ENCODING

# =====

# Create an instance of LabelEncoder to convert text labels into numbers.

le = LabelEncoder()

# Loop through all columns that have text ('object') or categorical data types.
# This converts every text feature in the dataset into numerical values automatically.
for col in df.select_dtypes(include=['object', 'category']).columns:
    df[col] = le.fit_transform(df[col])


# =====

# 4. SEPARATE FEATURES (INPUTS) AND TARGET (OUTPUT)

# =====

# Extract 'Health Condition' as our target variable (y/output) that the model will predict.

output = df['Health Condition']

# Drop 'Health Condition' from the dataframe so the remaining columns form our features (X/input).

inputt = df.drop('Health Condition', axis=1)

```

```
# Drop 'User ID' because ID numbers are unique identifiers and carry no useful pattern for prediction.
```

```
inputt = inputt.drop('User ID', axis=1)
```

```
# =====
```

```
# 5. FEATURE SCALING
```

```
# =====
```

```
# Create a MinMaxScaler instance to scale numeric inputs between 0 and 1.
```

```
scaler = MinMaxScaler()
```

```
# Transform all input features so that all values fit into the [0, 1] range.
```

```
# In this code, it normalizes all feature variables for stable neural network training.
```

```
inputt = scaler.fit_transform(inputt)
```

```
# =====
```

```
# 6. SPLIT DATA INTO TRAIN AND TEST SETS
```

```
# =====
```

```
# Split data into 80% training and 20% testing sets using random_state=42 for reproducible results.
```

```
# 'X_train' & 'y_train' are used to train the model, while 'X_test' & 'y_test' evaluate performance.
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    inputt, output, test_size=0.2, random_state=42  
)
```

```
# Print the training features to check the shape and scaled values.
```

```
print(X_train)
```



```

# Dynamically count the number of input feature columns.

# This gives the exact number of features feeding into the neural network's input layer.
num_cols = inputt.shape[1]

# Print the total number of input features.
print(num_cols)

# Print the training target labels to verify output values.
print(y_train)


# =====

# 7. BUILD THE NEURAL NETWORK

# =====

# Initialize a Sequential neural network model with three Dense layers.

# - Layer 1 ("rohim"): First hidden layer with 8 neurons, using 'relu' activation, receiving dynamic input size
(num_cols).

# - Layer 2 ("reshad"): Second hidden layer with 6 neurons and 'relu' activation to capture complex features.

# - Layer 3 ("uasaed"): Output layer with 4 neurons and 'softmax' activation to output probabilities for 4 target
classes.

model = Sequential([
    Dense(8, activation='relu', input_shape=(num_cols,)), name="rohim"),
    Dense(6, activation='relu', name="reshad"),
    Dense(4, activation='softmax', name="uasaed")
])

# Print a summary of the model architecture showing layer names, output shapes, and total trainable parameters.
print(model.summary())

```

```

# =====

# 8. COMPILE AND TRAIN THE MODEL

# =====

# Set up the Adam optimizer with a learning rate of 0.001 to control step size during gradient descent.
optimizer = Adam(learning_rate=0.001)

# Compile the model by configuring the optimizer, loss function, and performance metric.
# 'sparse_categorical_crossentropy' is used because targets are integer-encoded multi-class labels.
model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the neural network on training data for 2 epochs (passes over the data).
# 'verbose=2' displays one line of progress logs per epoch.
model.fit(X_train, y_train, epochs=2, verbose=2)

# =====

# 9. EVALUATE MODEL PERFORMANCE

# =====

# Evaluate loss and accuracy on unseen test data to verify model performance.
loss, accuracy = model.evaluate(X_test, y_test)

```

```
# Print the final test accuracy percentage achieved by the model on test data.
```

```
print("Test Accuracy:", accuracy)
```

CNN

```
# =====
```

```
# SECTION 1: IMPORTS AND INITIAL IMAGE INSPECTION
```

```
# =====
```

```
# Import PyTorch core library for deep learning operations
```

```
import torch
```

```
# Import neural network modules (layers, loss functions, etc.)
```

```
import torch.nn as nn
```

```
# Import optimization algorithms (Adam, SGD, etc.)
```

```
import torch.optim as optim
```

```
# Import torchvision for image computer vision utilities
```

```
import torchvision
```

```
# Import datasets, pre-trained models, and image transformation functions
```

```
from torchvision import datasets, models, transforms
```

```
# Import DataLoader to batch and shuffle datasets efficiently
```

```
from torch.utils.data import DataLoader
```

```
# Import PIL to load and read image files
```

```
from PIL import Image
```

```

# Open a sample image file from the specified Kaggle path

img =
Image.open("/kaggle/input/datasets/nahidhasangb/shutki/shutkiDATA/test/SliceHilsa/SingleSliceHilsa0001.jpg")


# Print the image dimensions in (width, height) format

print(img.size)


# Print the color format of the image (e.g., RGB, L for grayscale)

print(img.mode)


# Import NumPy library for array operations

import numpy as np


# Convert the PIL image into a numerical NumPy array

img = np.array(img)


# Print the shape of the image array (Height, Width, Channels)

print(img.shape)


# =====

# SECTION 2: IMAGE TRANSFORMATIONS AND DATASET PREPARATION

# =====


# Define transformations for training images to augment and normalize data

train_transform = transforms.Compose([

    # Resize images to 32x32 pixels to match network input shape

    transforms.Resize((32, 32)),

    # Randomly flip images horizontally to increase data diversity

```

```

transforms.RandomHorizontalFlip(),

# Randomly rotate images up to 10 degrees
transforms.RandomRotation(10),

# Convert image pixels into PyTorch tensor format (scales values to 0-1)
transforms.ToTensor(),

# Normalize image pixel values using ImageNet mean and standard deviation
transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
])

# Define transformations for testing images (no augmentations, only preprocessing)
test_transform = transforms.Compose([
    # Resize images to 32x32 pixels
    transforms.Resize((32, 32)),

    # Convert image pixels into PyTorch tensor format
    transforms.ToTensor(),

    # Normalize image pixel values using ImageNet mean and standard deviation
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

# Load training images from the directory and apply train transformations
train_dataset = datasets.ImageFolder(
    "/kaggle/input/datasets/nahidhasangb/shutki/shutkiDATA/train",
    transform=train_transform

```

```
)
```

```
# Load test images from the directory and apply test transformations
```

```
test_dataset = datasets.ImageFolder(  
    "/kaggle/input/datasets/nahidhasangb/shutki/shutkiDATA/test",  
    transform=test_transform  
)
```

```
# Print the class names detected from folder names
```

```
print(train_dataset.classes)
```

```
# Create DataLoader for training set (batches of 32, shuffled randomly)
```

```
train_loader = DataLoader(  
    train_dataset,  
    batch_size=32,  
    shuffle=True  
)
```

```
# Create DataLoader for test set (batches of 32, not shuffled)
```

```
test_loader = DataLoader(  
    test_dataset,  
    batch_size=32,  
    shuffle=False  
)
```

```
# =====
```

```
# SECTION 3: FEDERATED LEARNING DATA SPLITTING (FASHION-MNIST)
```

```
# =====
```

```
# Import random_split tool to split datasets into smaller subsets
from torch.utils.data import DataLoader, random_split

# Define total number of simulated clients for federated learning
clients = 20

# Set a fixed random seed for reproducible dataset splits
generator = torch.Generator().manual_seed(42)

# Define normalization parameters for FashionMNIST dataset
FMNIST_MEAN = (0.2860,)
FMNIST_STD = (0.3530,)

# Transformation pipeline to prepare FashionMNIST images for 3-channel models
fmnist_transform = transforms.Compose([
    # Resize image to 32 pixels
    transforms.Resize(32),
    # Convert 1-channel grayscale image to 3 channels (RGB compatible)
    transforms.Grayscale(num_output_channels=3),
    # Convert image to PyTorch Tensor
    transforms.ToTensor(),
    # Normalize using FashionMNIST mean and standard deviation
    transforms.Normalize(FMNIST_MEAN, FMNIST_STD)
])

# Download and load official FashionMNIST training dataset
fmnist_full_train = torchvision.datasets.FashionMNIST(
    root='./data',
```

```
train=True,  
  
download=True,  
  
transform=fmnist_transform  
)
```

```
# Download and load official FashionMNIST test dataset
```

```
fmnist_official_test = torchvision.datasets.FashionMNIST(  
  
    root='./data',  
  
    train=False,  
  
    download=True,  
  
    transform=fmnist_transform  
)
```

```
# Define sizes to separate main training data and attack split data
```

```
main_train_len = 59500  
  
attack_split_len = 500
```

```
# Create seed generator for reproducibility
```

```
seed_generator = torch.Generator().manual_seed(42)
```

```
# Split full FashionMNIST train set into main train set (59500) and attack set (500)
```

```
fmnist_train_part, fmnist_attack_part = random_split(  
  
    fmnist_full_train,  
  
    [main_train_len, attack_split_len],  
  
    generator=seed_generator  
)
```

```
# Calculate total length of main training partition
```

```
fmnist_train_part_len = len(fmnist_train_part)
```



```
# Calculate equal base size of samples per client
```

```
part_size = fmnist_train_part_len // clients
```

```
# Create a list containing equal dataset partition sizes for each client
```

```
sizes = [part_size] * clients
```

```
# Add remaining samples to the last client so total sum equals total samples
```

```
sizes[-1] += fmnist_train_part_len - sum(sizes)
```

```
# Split the training data among 20 virtual clients
```

```
fmnist_train_part_subsets = random_split(fmnist_train_part, sizes, generator=generator)
```

```
# Print sample count for Client 0
```

```
print(len(fmnist_train_part_subsets[0]))
```

```
# Calculate total length of test partition
```

```
fmnist_test_part_len = len(fmnist_official_test)
```

```
# Calculate base test partition size per client
```

```
part_size = fmnist_test_part_len // clients
```

```
# Create list of partition sizes for test set split
```

```
sizes = [part_size] * clients
```

```
# Add remaining test samples to the last client's size
```

```
sizes[-1] += fmnist_test_part_len - sum(sizes)
```

```
# Split official test set among 20 virtual clients
```

```

fmnist_test_part_subsets = random_split(fmnist_official_test, sizes, generator=generator)

# Print sample count for Client 0 test partition
print(len(fmnist_test_part_subsets[0]))

# Prepare DataLoader for Client index 10 (11th client) training data
fmnist_train_loader = DataLoader(fmnist_train_part_subsets[10], batch_size=32, shuffle=True)

# Prepare DataLoader for Client index 10 testing data
fmnist_test_loader = DataLoader(fmnist_test_part_subsets[10], batch_size=32, shuffle=False)

# =====
# SECTION 4: RESNET-18 FINE-TUNING AND TRAINING
# =====

# Import ResNet models from torchvision and torchinfo for model summary
from torchvision import models
from torchinfo import summary

# Initialize an unweighted ResNet18 model architecture
model2 = models.resnet18(weights=None)

# Freeze initial layers and keep deeper layers trainable
for idx, child in enumerate(model2.children()):
    if idx < 2:
        # Freeze parameters in first two layers (prevent weight update)
        for param in child.parameters():
            param.requires_grad = False

```

```
else:

    # Allow parameters in remaining layers to update during training
    for param in child.parameters():
        param.requires_grad = True

# Replace final linear layer to output 10 classes instead of default 1000
model2.fc = nn.Linear(model2.fc.in_features, 10)

# Display network architecture and total trainable parameters summary
summary(model2, input_size=(1, 3, 32, 32))

# Define CrossEntropyLoss function suitable for multi-class classification
criterion = nn.CrossEntropyLoss()

# Define Adam optimizer to update model parameters with learning rate 0.001
optimizer = optim.Adam(model2.parameters(), lr=0.001)

# Set total training epochs
epochs = 2

# Training loop over specified epochs
for epoch in range(epochs):

    # Set model to training mode
    model2.train()

    # Track accumulated loss, correct predictions, and total samples
    running_loss = 0
    correct = 0
    total = 0
```

```
# Iterate over batches of images and labels in Client 10's loader
for images, labels in fmnist_train_loader:

    # Clear gradients from previous iteration
    optimizer.zero_grad()

    # Forward pass: compute predicted outputs from input images
    outputs = model2(images)

    # Calculate loss between predictions and true labels
    loss = criterion(outputs, labels)

    # Backward pass: compute loss gradients with respect to model parameters
    loss.backward()

    # Update model parameters using gradients
    optimizer.step()

    # Add batch loss value to running total
    running_loss += loss.item()

    # Get predicted class indices with highest probability
    _, predicted = torch.max(outputs, 1)

    # Increment total sample count
    total += labels.size(0)

    # Count correctly predicted samples
    correct += (predicted == labels).sum().item()
```

```

# Calculate overall training accuracy percentage

train_acc = 100 * correct / total


# Print epoch progress, average loss, and training accuracy
print(
    f"Epoch {epoch+1}/{epochs}\n"
    f"Loss={running_loss/len(fmnist_train_loader):.4f}\n"
    f"Accuracy={train_acc:.2f}%"
)


# Set model to evaluation mode (disables Dropout and BatchNorm updates)
model2.eval()


# Reset evaluation counters
correct = 0
total = 0


# Disable gradient calculation to save memory during evaluation
with torch.no_grad():
    # Iterate over client test dataset
    for images, labels in fmnist_test_loader:
        # Get model outputs
        outputs = model2(images)

        # Get predicted class indices
        _, predicted = torch.max(outputs, 1)

        # Accumulate total samples and correct predictions

```

```

total += labels.size(0)

correct += (predicted == labels).sum().item()

# Calculate overall test accuracy percentage
test_acc = 100 * correct / total

# Print test accuracy result
print(f"Test Accuracy: {test_acc:.2f}%")

# =====

# SECTION 5: SIMPLE MLP (MULTI-LAYER PERCEPTRON) MODEL

# =====

# Define a simple fully connected neural network (MLP)
modelSmallC = nn.Sequential(

    # Flatten 3D image tensor into 1D array (3 channels * 32 * 32 = 3072 input nodes)
    nn.Flatten(),

    # Layer 1: Linear layer mapping 3072 inputs to 8 features
    nn.Linear(3072, 8),

    # ReLU activation function for non-linearity
    nn.ReLU(),

    # Layer 2: Linear layer mapping 8 features to 4 features
    nn.Linear(8, 4),

    # ReLU activation function
    nn.ReLU(),

```

```
# Output Layer: Linear layer mapping 4 features to 2 output classes  
nn.Linear(4, 2)  
)
```

```
# Print architecture summary of simple MLP model  
summary(modelSmallC, input_size=(1, 3, 32, 32))
```

```
# Set loss function for MLP  
criterion = nn.CrossEntropyLoss()
```

```
# Set optimizer for MLP model  
optimizer = optim.Adam(modelSmallC.parameters(), lr=0.001)
```

```
# Set epoch count for training  
epochs = 5
```

```
# Training loop for simple MLP model
```

```
for epoch in range(epochs):
```

```
    # Set model to training mode  
    modelSmallC.train()
```

```
    running_loss = 0
```

```
    correct = 0
```

```
    total = 0
```

```
# Train on Shutki image training loader
```

```
for images, labels in train_loader:
```

```
    # Reset gradients
```

```
    optimizer.zero_grad()
```

```

# Forward pass
outputs = modelSmallC(images)

# Calculate loss
loss = criterion(outputs, labels)

# Compute gradients
loss.backward()

# Update weights
optimizer.step()

# Update metrics
running_loss += loss.item()
_, predicted = torch.max(outputs, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()

# Compute training accuracy
train_acc = 100 * correct / total

# Print training loss and accuracy
print(
    f"Epoch {epoch+1}/{epochs}\n"
    f"Loss={running_loss/len(train_loader):.4f} "
    f"Accuracy={train_acc:.2f}%"
)

```



```

# Set model to evaluation mode

modelSmallC.eval()

correct = 0

total = 0

# Test simple MLP model without gradients
with torch.no_grad():

    for images, labels in test_loader:

        outputs = modelSmallC(images)

        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)

        correct += (predicted == labels).sum().item()

# Compute final test accuracy

test_acc = 100 * correct / total

# Print final test accuracy

print(f"Test Accuracy: {test_acc:.2f}%")

# =====

# SECTION 6: CUSTOM CNN (CONVOLUTIONAL NEURAL NETWORK) MODEL ARCHITECTURE

# =====

# Define a custom Convolutional Neural Network (CNN) architecture

model = nn.Sequential(

    # First Conv Layer: 3 input channels (RGB), 32 filters, 3x3 kernel, padding=1

    nn.Conv2d(3, 32, kernel_size=3, padding=1),

```

```

# ReLU Activation

nn.ReLU(),

# Max Pooling: reduces spatial dimensions by a factor of 4

nn.MaxPool2d(4),


# Second Conv Layer: 32 input channels, 64 output filters, 3x3 kernel

nn.Conv2d(32, 64, kernel_size=3, padding=1),

# ReLU Activation

nn.ReLU(),

# Max Pooling: reduces spatial dimensions by a factor of 2

nn.MaxPool2d(2),


# Flatten 2D feature maps into 1D vector for dense layers

nn.Flatten(),


# Fully Connected Layer: maps flattened features (1024) to 128 nodes

nn.Linear(64 * 4 * 4, 128),

# ReLU Activation

nn.ReLU(),


# Output Layer: maps 128 features to 10 output class logits

nn.Linear(128, 10)

)


# Print CNN architecture summary

summary(model, input_size=(1, 3, 32, 32))


# =====

```

```
# SECTION 7: TENSORFLOW / KERAS BINARY CLASSIFICATION ON DIABETES DATASET
```

```
# =====
```

```
# Import Pandas for CSV reading
```

```
import pandas as pd
```

```
# Import train-test splitter from Scikit-Learn
```

```
from sklearn.model_selection import train_test_split
```

```
# Import MinMaxScaler for feature scaling
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
# Import TensorFlow and Keras components
```

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
from tensorflow.keras.optimizers import Adam
```

```
# Read Diabetes CSV dataset into a Pandas DataFrame
```

```
df = pd.read_csv('/kaggle/input/datasets/nahidhasangb/diabetes-sumon-faisal/Db.csv')
```

```
# Select all feature columns except last 5
```

```
X = df.iloc[:, :-5]
```

```
# Select last column as target label (diabetes outcome)
```

```
y = df.iloc[:, -1]
```

```
# Initialize MinMaxScaler to scale features into range [0, 1]
```

```
scaler = MinMaxScaler()
```

```
# Fit scaler and transform feature values
```

```
X = scaler.fit_transform(X)
```

```
# Split dataset into 80% training data and 20% testing data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Define Keras Sequential model for binary classification
```

```
model = Sequential([
```

```
    # Hidden Layer 1: 8 neurons, ReLU activation, 4 input features
```

```
    Dense(8, activation='relu', input_shape=(4,), name="rohim"),
```

```
    # Hidden Layer 2: 2 neurons, ReLU activation
```

```
    Dense(2, activation='relu', name="reshad"),
```

```
    # Output Layer: 1 neuron, Sigmoid activation for probability (0 to 1)
```

```
    Dense(1, activation='sigmoid', name="uasaed")
```

```
])
```

```
# Print model architectural summary
```

```
print(model.summary())
```

```
# Initialize Adam optimizer with learning rate 0.001
```

```
optimizer = Adam(learning_rate=0.001)
```

```
# Compile model with binary cross-entropy loss and accuracy metric
```

```
model.compile(
```

```
    optimizer=optimizer,
```

```
    loss='binary_crossentropy',
```

```
    metrics=['accuracy']
```

```
)
```

```
# Train Keras model for 10 epochs
```

```
model.fit(X_train, y_train, epochs=10, verbose=2)
```

```
# Evaluate model performance on test set
```

```
loss, accuracy = model.evaluate(X_test, y_test)
```

```
# Print final test accuracy
```

```
print("Test Accuracy:", accuracy)
```

```
# =====
```

```
# SECTION 8: HYPERPARAMETER OPTIMIZATION USING GENETIC ALGORITHM (GA)
```

```
# =====
```

```
# Import random module for genetic operations
```

```
import random
```

```
# Define chromosome length (number of layers in network)
```

```
csize = 8
```

```
# Define initial population size (number of network candidates)
```

```
initPopSize = 6
```

```
# Define number of generations to evolve
```

```
numOfgen = 3
```

```
# Function to generate a random chromosome (list of layer neuron counts)
```

```
def genChrom():
```

```
chrom = []  
  
for i in range(csize):  
    # Generate random neuron size between 1 and 8  
    k = random.randint(1, 8)  
    chrom.append(k)  
  
return chrom
```

Crossover function combining two parent chromosomes to produce two children

```
def crossover(p1, p2):  
    # Select random crossover point between index 3 and 8  
    point = random.randint(3, 8)  
    print('point', point)  
    # Swap tail sections of parents  
    c1 = p1[:point] + p2[point:]  
    c2 = p2[:point] + p1[point:]  
    return c1, c2
```

Mutation function introducing random parameter variation

```
def mutate(chrom):  
    # Pick a random index point to mutate  
    point = random.randint(0, 7)  
    print('point', point)  
    # Pick a new random neuron value  
    rval = random.randint(1, 8)  
    # Update chromosome at mutation index  
    chrom[point] = rval  
    return chrom
```

Fitness function evaluating how well a chromosome architecture performs

```

def fitness(chrom, X_train, y_train, X_test, y_test):

    # Clear previous Keras session memory to avoid clutter

    tf.keras.backend.clear_session()

    # Initialize Sequential Keras model

    model = Sequential()

    # Add input layer using first gene value as neuron count

    model.add(Dense(chrom[0], activation='relu', input_shape=(4,), name="layer_0"))

    # Add hidden layers corresponding to middle genes

    for i in range(1, len(chrom) - 1):

        model.add(Dense(chrom[i], activation='relu', name=f"layer_{i}"))

    # Add binary output layer

    model.add(Dense(1, activation='sigmoid', name="output"))

    # Set Adam optimizer

    optimizer = Adam(learning_rate=0.001)

    # Compile model

    model.compile(

        optimizer=optimizer,

        loss='binary_crossentropy',

        metrics=['accuracy']

    )

    # Train candidate model silently for 2 epochs

    model.fit(X_train, y_train, epochs=2, verbose=0)

```

```

# Evaluate model loss on test set

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)


# Higher fitness value returned for lower loss score

return 1 - loss


# Initialize population list and fitness list

population = []

f = []


# Generate initial random population and calculate initial fitness scores

for i in range(initPopSize):

    chrom = genChrom()

    f.append(fitness(chrom, X_train, y_train, X_test, y_test))

    population.append(chrom)


# Print initial population and fitness values

print(population)

print(f)


# Genetic Algorithm evolution loop across generations

for i in range(numOfgen):

    newpop = []

    print('-----', i)


# Build new generation until target population size is reached

while len(newpop) < 6:

    # Identify top 2 fittest chromosome indices

```



```
top2_idx = sorted(range(len(f)), key=lambda idx: f[idx], reverse=True)[:2]
```

```
# Select parents with highest fitness
```

```
p1 = population[top2_idx[0]]
```

```
p2 = population[top2_idx[1]]
```

```
# Perform crossover to create 2 offspring
```

```
c1, c2 = crossover(p1, p2)
```

```
# Apply mutation to both offspring
```

```
c1 = mutate(c1)
```

```
c2 = mutate(c2)
```

```
# Add offspring to new population
```

```
newpop.append(c1)
```

```
newpop.append(c2)
```

```
# Replace old population with evolved population
```

```
population = newpop
```

```
f = []
```

```
# Calculate fitness for new population members
```

```
for j in range(len(population)):
```

```
    f.append(fitness(population[j], X_train, y_train, X_test, y_test))
```

```
# =====
```

```
# SECTION 9: KERAS MODEL REGULARIZATION (L2 REGULARIZATION & DROPOUT)
```

```
# =====
```

```
# Import L2 regularizer from Keras
```

```
from tensorflow.keras.regularizers import l2
```

```
# Import Dropout layer from Keras
```

```
from tensorflow.keras.layers import Dropout
```

```
# Model example using L2 Regularization to prevent overfitting
```

```
model_l2 = Sequential([
```

```
    # Layer 1 with L2 kernel penalty of 0.01
```

```
    Dense(64, activation='relu', input_shape=(4,)), kernel_regularizer=l2(0.01)),
```

```
    # Layer 2 with L2 kernel penalty of 0.01
```

```
    Dense(32, activation='relu', kernel_regularizer=l2(0.01)),
```

```
    # Sigmoid Output Layer
```

```
    Dense(1, activation='sigmoid')
```

```
])
```

```
# Model example using Dropout to prevent overfitting
```

```
model_dropout = Sequential([
```

```
    # Input Dense layer with 64 units
```

```
    Dense(64, activation='relu', input_shape=(4,)),
```

```
    # Dropout layer: randomly deactivates 30% of neurons during training
```

```
    Dropout(0.3),
```

```
    # Second Dense layer with 32 units
```

```
    Dense(32, activation='relu'),
```

```
    # Dropout layer: randomly deactivates 30% of neurons during training
```

```
    Dropout(0.3),
```

```
    # Sigmoid Output layer
```

```
    Dense(1, activation='sigmoid')
```

1)